LogStruct: learned prior-anchored gene-graph smoothing for interpretable omics prediction

Draft status: scaffold only. Do not submit until every result placeholder is replaced by a logged run under `results/`.

Abstract

High-dimensional omics prediction often benefits from biological prior knowledge, but existing network-regularized linear models typically treat the gene graph as fixed. We evaluate LogStruct, a sklearn-compatible PyTorch estimator that learns a Bernoulli-parameterized feature graph anchored to a biological prior, uses the learned graph as an explicit feature-smoothing operator, and retains linear-model interpretability through effective coefficients. This draft currently reports only code-audit and synthetic smoke test results. Bulk, single-cell, drug-response, and biological validation results remain to be run before any empirical claims are made.

1. Introduction

Genomic and single-cell prediction problems often have many more molecular features than independently sampled biological units. Sparse linear models are attractive in this regime because they are simple, reproducible, and inspectable, but ordinary lasso, ridge, and elastic net ignore curated knowledge about gene interactions, pathways, and regulatory structure. Fixed-graph Laplacian methods address this by encouraging neighboring genes to have similar coefficients, but the prior graph is often noisy, incomplete, or mismatched to the phenotype.

LogStruct tests a middle ground. It keeps the estimator linear after a learned feature-smoothing operator, while allowing the gene graph to deviate from a prior through a KL-anchored Bernoulli adjacency. This is designed for two regimes: bulk transcriptomics, where sample size is small and prior structure may improve sample efficiency, and single-cell data, where rare cell types or perturbations have limited support even when the total number of cells is large.

The planned contributions are:

1. A precise implementation audit of LogStruct's smoothing, Laplacian, and KL terms, including effective coefficients for interpretation.
2. A controlled empirical comparison against fixed-graph Laplacian models, GELnet-style penalties, classical baselines, gradient boosting, MLPs, and GNNs.
3. Biological validation of learned effective coefficients and graph changes using pathway enrichment, marker recovery, gained/lost edges, and regulatory recovery in perturb-seq.

2. Related work

See `paper/related_work.md`. In brief, LogStruct is positioned between fixed-graph sparse linear methods such as Li and Li, Logit-Lapnet, glmgraph, and GELnet, and learned-graph neural methods such as DIAL-GNN. It should be framed as an incremental but testable combination rather than a new modeling paradigm.

3. Method

Let `X in R^(n x p)` be a sample-by-feature matrix and let `A0 in [0,1]^(p x p)` be a biological prior adjacency. LogStruct learns adjacency logits `L` and defines:

```
A = zero_diag(0.5 * (sigmoid(L / T) + sigmoid(L / T)^T))
S(A) = alpha I + beta rownorm(A)
Y_hat = X S(A) W^T + b
```

The objective for classification is:

```
CE(Y, Y_hat)
+ lambda_en EN(W)
+ lambda_smooth trace(W L_A W^T)
+ lambda_kl KL(q(A) || p(A0))
```

Regression replaces cross-entropy with mean squared error. `EN(W)` is the elastic-net penalty. `L_A = D_A - A` is the graph Laplacian. The KL term is a Bernoulli KL over adjacency probabilities. This branch adds an `offdiag_mean` KL reduction to avoid `O(p^2)` scaling of the penalty.

For interpretation, the raw head weights `W` are not the coefficients on the original feature space. The effective coefficients are:

```
W_effective = W S(A)^T
```

All biological interpretation in this paper must use `effective_coef_`, not raw `coef_`.

4. Experimental setup

Planned datasets:

- METABRIC PAM50 breast cancer subtype classification.
- TCGA pan-cancer classification, using the UCI subset for fast shakedown and UCSC Xena for the full 33-class run.
- GDSC drug-response regression.
- GTEx broad tissue classification.
- Tabula Sapiens immune-compartment cell-type classification with donor-held-out splits.

- Norman 2019 perturb-seq perturbation identity classification and regulatory recovery.

All experiments will use fixed train/validation/test splits, train-only preprocessing, bootstrap confidence intervals, paired significance tests, and per-fold prediction logs. No result enters this manuscript unless it appears in a `results.json` file with seed, hyperparameters, git SHA, wall-clock time, and hardware record.

5. Results

5.1 Code audit and smoke test

The code audit identified and fixed three implementation issues: missing effective coefficients, temperature-distorted prior probabilities in the KL anchor, and missing STRING alias resolution. It also added unthresholded `adjacency_raw_` to distinguish the prediction-time graph from the thresholded interpretation graph.

The synthetic smoke test in `results/smoke/results.json` passed with balanced accuracy 0.773, macro-F1 0.777, and OvR AUROC 0.965. This verifies the training pipeline only and is not a biological result.

5.2 Main comparison

Pending real-data runs. Table placeholder: `paper/tables/table1_main_results.md`.

5.3 Prior sensitivity

Pending real-data runs.

5.4 Frozen versus learned adjacency

Pending real-data runs.

5.5 Smoothing-operator ablation

Pending real-data runs.

5.6 Sample efficiency

Pending real-data runs.

5.7 Single-cell cell-type classification

Pending real-data runs.

5.8 Perturbation identity and regulatory recovery

Pending real-data runs.

5.9 Biological interpretation

Pending real-data runs.

6. Discussion

The audit already clarifies two important limitations. First, dense adjacency learning materializes `p x p` matrices and therefore has an `O(p^2)` memory ceiling. Second, KL scaling is not a minor numerical detail: the legacy summed KL can dominate at large feature counts, so the paper must report the chosen KL normalization and sensitivity to `lambda_kl`.

The empirical discussion will be written only after logged real-data runs. If LogStruct underperforms fixed-graph or classical baselines, that will be reported directly.

7. Conclusion

Pending real-data runs.

Availability

Code branch: `experiments/v1`. A release, Zenodo DOI, and reproducible Table 1 notebook remain pending.